

PREPARED BY

Fatima Noor-ul-Imran(23-BCS-033)
Muneeba Jamil(23-BCS-066)

INSTRUCTOR Sir Shoaib

COMPUTER ARCHITECTURE REPORT

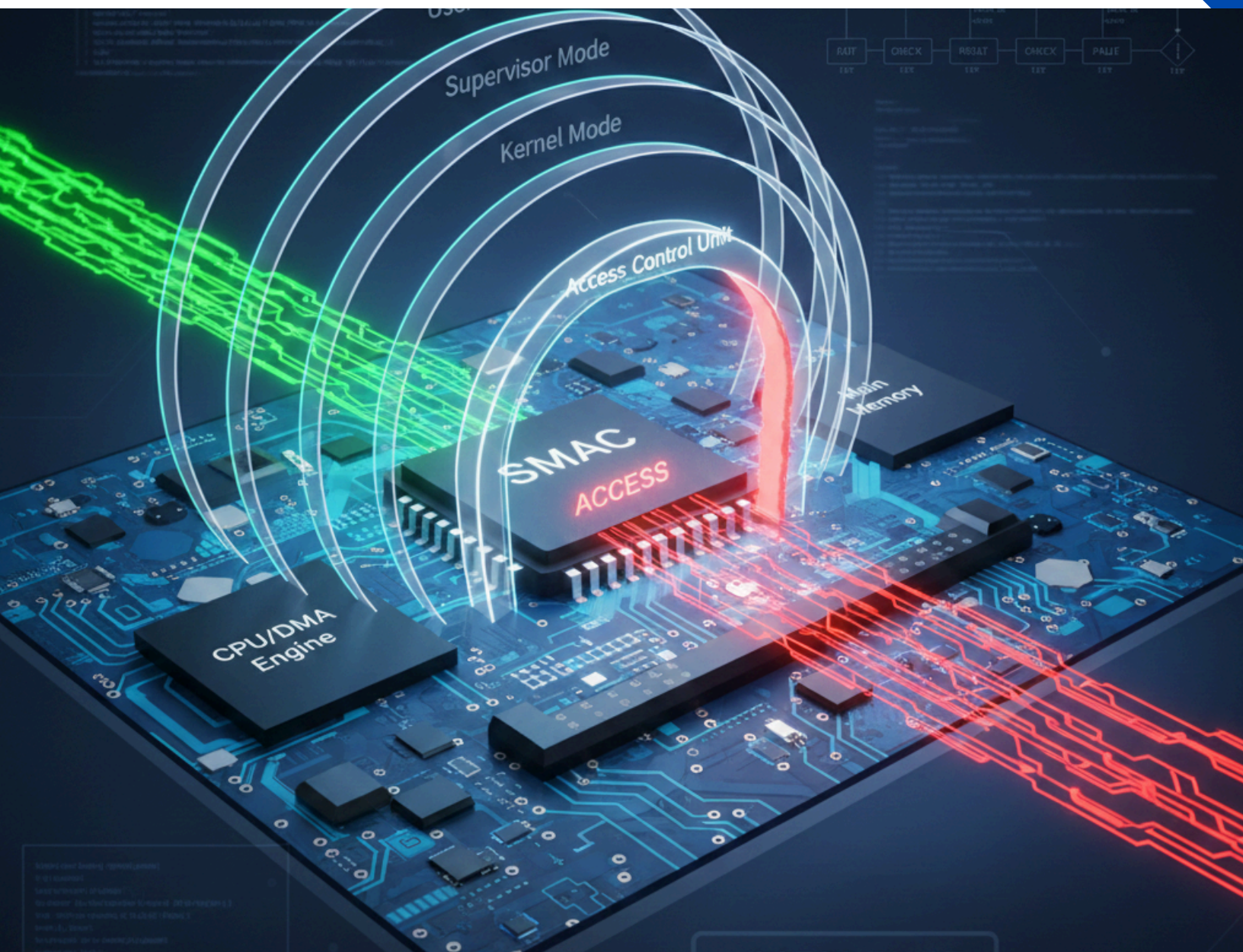


Table of Contents

Abstract	2
Introduction	2
Importance Of Memory Protection	2
Direct Memory Access (DMA) Attacks	2
OS Vs User Isolation.....	2
Why Hardware Enforcement is Better than Software	3
Problem Statement	3
System Requirement	3
Functional Requirement	3
Non- Functional Requirement	4
Requirement Traceability.....	4
Architecture Overview.....	5
Functional Module Description	5
Privilege Decoder	5
Region Decoder	6
Access Control Unit	6
Fault Logger.....	6
Top Level SMAC Module	7
FSM Design	7
Access Control Algorithm.....	8
Implementation Detail	8
Verilog HDL Implementation.....	8
Modular Coding Approach	8
Combinational vs Sequential Logic	8
Clock Usage	8
Synthesizable Design Style	9
Latch Free Design	9
Blocking vs Non-Blocking Assignments	9
Simulation & Verification.....	9
Simulation Environment	9
Test Case Description	9
Test Case Table.....	9
Simulation Result.....	10
Waveform Analysis	11
Console Output Verification	11
Result & Discussion	12
Limitation & Future Enhancement	12
Conclusion.....	12
Reference.....	13

SECURE MEMORY ACCESS CONTROLLER

1. Abstract

This project presents the design and implementation of a Secure Memory Access Controller (SMAC) that enforces hardware-level memory protection using privilege modes and read/write/execute permissions. The system divides memory into multiple regions and validates each memory access request based on address range, privilege level, and access type. Unauthorized accesses are blocked and reported using fault signaling. The design is implemented in Verilog HDL and verified using ModelSim simulation. The proposed SMAC models security mechanisms used in modern processors such as ARM and RISC-V.

2. Introduction

2.1 Importance of Memory Protection

Memory protection is a fundamental requirement in modern computer systems to ensure system stability, security, and reliability. It prevents unauthorized programs from accessing or modifying memory regions that belong to other applications or the operating system. Without memory protection, a single faulty or malicious program could corrupt critical system data, leading to crashes, data leaks, or complete system compromise. By enforcing strict access rules at the hardware level, memory protection ensures isolation between user applications and privileged system components.

2.2 Direct Memory Access (DMA) Attacks

Direct Memory Access (DMA) allows hardware devices such as network cards, GPUs, or external interfaces (e.g., Thunderbolt or PCIe) to access system memory directly without CPU intervention. While DMA improves performance, it also introduces serious security risks. Malicious or compromised devices can exploit DMA to read or modify sensitive memory regions, bypassing software-based security checks. These DMA attacks can extract encryption keys, alter kernel data structures, or inject malicious code. Therefore, controlling and filtering DMA memory requests using hardware-based protection mechanisms is critical for system security.

2.3 OS vs User Isolation

Operating systems operate in multiple privilege levels to maintain control over system resources. User applications run in user mode with limited privileges, while the operating system kernel runs in a highly privileged mode with full access to hardware and memory. This separation ensures that user programs cannot directly interfere with OS services or kernel memory. OS vs user isolation protects the system from accidental bugs in applications as well as intentional attacks, ensuring that only trusted kernel code can manage critical resources such as memory management, process scheduling, and device control.

2.4 Why Hardware Enforcement is Better than Software

Software-based security mechanisms rely on the correctness of code execution and can be bypassed if vulnerabilities exist. In contrast, hardware-enforced memory protection operates independently of software and cannot be disabled or bypassed by compromised programs. Hardware enforcement checks every memory access in real time, ensuring consistent and reliable protection with minimal performance overhead. This approach provides stronger security guarantees, faster response to illegal accesses, and a trusted foundation upon which secure operating systems are built.

3. Problem Statement

Software-only memory protection is vulnerable to attacks and bugs. Without hardware enforcement, user applications may access or execute privileged memory regions. Therefore, a hardware-based Secure Memory Access Controller is required to ensure isolation and system security.

4. System Requirements

4.1 Functional Requirements

Requirement ID	Requirement Description
FR-1	The system shall enforce privilege-based access control by validating every memory access request against the current execution mode (User, Supervisor, Kernel).
FR-2	The system shall divide the physical memory address space into multiple protected regions, each associated with a required privilege level and access permissions.
FR-3	The system shall enforce execute permission by blocking instruction execution from non-executable memory regions, regardless of read or write permissions.
FR-4	The system shall detect illegal memory access attempts and generate an access fault signal when privilege or permission violations occur.
FR-5	The system shall log the address of the most recent illegal memory access to support debugging and security monitoring.

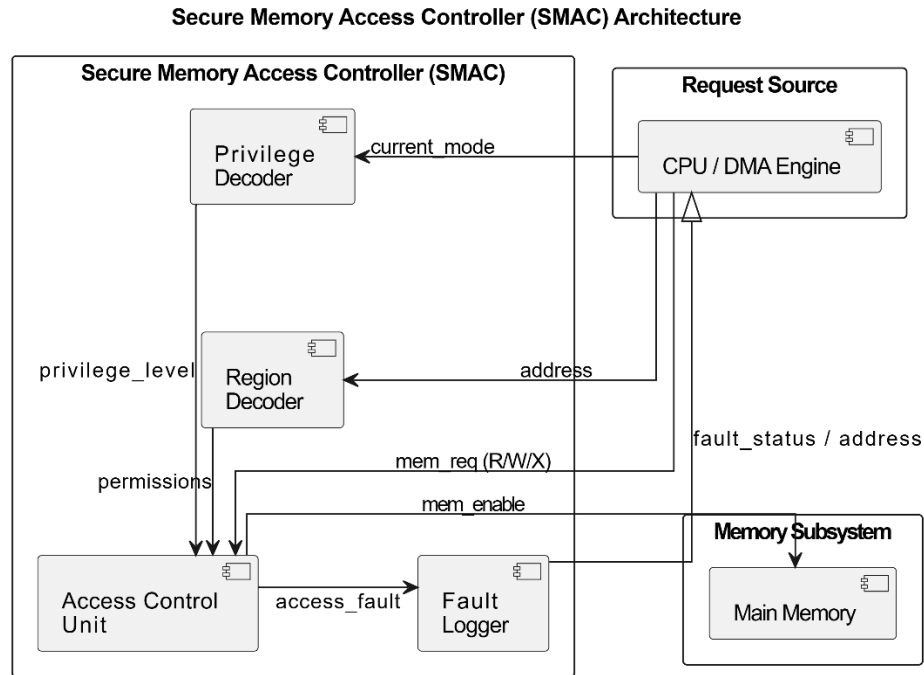
4.2 Non-Functional Requirements

Requirement ID	Requirement Description
NFR-1	The system shall perform memory access validation with minimal latency to avoid performance degradation during normal operation.
NFR-2	The system shall be implemented using a modular hardware architecture, separating privilege decoding, region decoding, access control, and fault logging logic.
NFR-3	The system architecture shall be scalable, allowing additional memory regions, permissions, or access control features to be integrated with minimal redesign.
NFR-4	The system shall be implemented using synthesizable Verilog HDL constructs, ensuring compatibility with standard simulation and synthesis tools.
NFR-5	The system shall support verification through simulation by providing observable outputs such as access enable and fault signals.

4.3 Requirement Traceability

Requirement ID	Implemented Module
FR-1	Privilege Decoder, Access Control
FR-2	Region Decoder
FR-3	Region Decoder, Access Control
FR-4	Access Control
FR-5	Fault Logger
NFR-2	Top-Level SMAC Architecture

5. Architecture Overview



The block diagram illustrates the architecture of the Secure Memory Access Controller (SMAC). A memory access request initiated by the CPU or a DMA-capable device is first evaluated by the SMAC before reaching the memory subsystem. The Privilege Decoder interprets the current execution mode of the requester, while the Region Decoder determines the memory region and its associated access permissions based on the requested address. The Access Control Unit compares the privilege level and requested access type (read, write, execute) against the region's protection policy. If the access is authorized, the memory enable signal is asserted, allowing the request to proceed to main memory. Otherwise, an access fault is generated and recorded by the Fault Logger, which reports the violation back to the CPU.

6. Functional Module Description

6.1 Privilege Decoder

Inputs:

`current_mode` – Indicates the current execution privilege mode (User, Supervisor, Kernel)

Outputs:

`privilege_level` – Encoded privilege level used for access comparison

Function:

The Privilege Decoder interprets the current execution mode of the requester and converts it into a standardized privilege level. This privilege level is used by the Access Control Unit to

determine whether the requester has sufficient authority to access a specific memory region. The module ensures consistent handling of privilege information across the system.

6.2 Region Decoder

Inputs:

address – Memory address requested by the CPU or DMA engine

Outputs:

required_mode – Minimum privilege level required to access the region

allow_read – Indicates whether read access is permitted

allow_write – Indicates whether write access is permitted

allow_execute – Indicates whether execute access is permitted

Function:

The Region Decoder maps the requested memory address to a predefined memory region and determines the corresponding protection policy. It specifies the minimum required privilege level and the allowed read, write, and execute permissions for that region. This module enforces region-based memory protection by associating each address range with its security attributes.

6.3 Access Control Unit

Inputs:

privilege_level – Current privilege of the requester

required_mode – Required privilege level for the target region

mem_read – Read request signal

mem_write – Write request signal

mem_execute – Execute request signal

allow_read, allow_write, allow_execute – Permission flags from Region Decoder

Outputs:

allow – Enables memory access if all checks pass

fault – Indicates an illegal access attempt

Function:

The Access Control Unit is the core security logic of the SMAC. It compares the requester's privilege level with the required privilege of the memory region and validates the requested access type against the region's permissions. If all conditions are satisfied, the memory access is allowed. Otherwise, an access fault is generated to block the request.

6.4 Fault Logger

Inputs:

clk – System clock

fault – Access violation signal

address – Address associated with the fault

Outputs:

last_fault_addr – Stores the address of the most recent illegal access

fault_valid – Indicates a valid logged fault

Function:

The Fault Logger captures and records illegal memory access attempts. Upon detection of an access fault, it stores the offending address and flags the occurrence. This module supports debugging, security monitoring, and post-event analysis by providing visibility into access violations.

6.5 Top-Level SMAC Module

Inputs:

clk – System clock

address – Requested memory address

current_mode – Execution privilege mode

mem_read, mem_write, mem_execute – Memory access control signals

Outputs:

mem_enable – Enables memory access when allowed

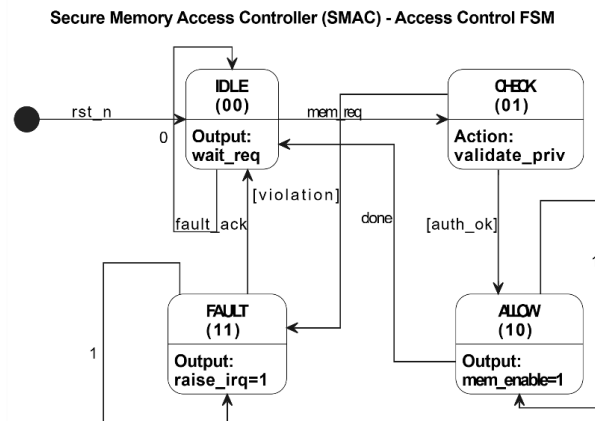
access_fault – Indicates an illegal access

fault_address – Address of the most recent fault

Function:

The Top-Level SMAC module integrates all functional blocks into a single cohesive system. It coordinates privilege decoding, region decoding, access validation, and fault logging to enforce hardware-based memory protection. This module serves as the interface between the requester and the memory subsystem, ensuring that only authorized accesses are permitted.

7. FSM DESIGN



The Secure Memory Access Controller operates as a simple and efficient finite state machine to control memory access requests. In the IDLE state, the controller waits for a memory access request. When a read, write, or execute request is detected, the FSM transitions to the CHECK state, where the address is decoded, the required privilege level is determined, and access permissions are validated. If the request satisfies both privilege and permission checks, the FSM transitions to the ALLOW state, enabling memory access. If any violation is detected, the FSM transitions to the FAULT state, where the access is blocked and the fault address is recorded. After completing either action, the FSM returns to the IDLE state to process the next request.

8. Access Control Algorithm:

```
if privilege < required_mode
    fault
else if execute and execute_not_allowed
    fault
else
    allow
```

9. Implementation Details

The Secure Memory Access Controller (SMAC) is implemented using Verilog Hardware Description Language (HDL). The design follows standard hardware engineering practices to ensure correctness, clarity, and synthesizability.

9.1 Verilog HDL Implementation

The entire system is described using Verilog HDL, a widely used hardware description language for modeling, simulation, and synthesis of digital systems. Verilog allows precise control over signal behavior and timing, making it suitable for implementing low-level hardware security mechanisms such as memory protection and access control.

9.2 Modular Coding Approach

The SMAC is designed using a modular architecture, where each functional block is implemented as an independent Verilog module. Separate modules are used for privilege decoding, region decoding, access control, fault logging, and top-level integration. This modular coding approach improves readability, simplifies debugging, and allows individual modules to be verified independently. It also enables scalability by allowing additional features to be integrated without modifying the entire design.

9.3 Combinational vs Sequential Logic

Most of the SMAC logic is implemented using combinational circuits, particularly in the privilege decoder, region decoder, and access control unit. These blocks evaluate memory access requests in a single cycle based on current inputs, ensuring low latency.

Sequential logic is used only where state retention is required. The fault logger module uses sequential logic to store the address of the most recent illegal access. This separation ensures efficient performance while maintaining necessary state information.

9.4 Clock Usage

The system clock is used exclusively in the fault logger module. The clocked logic ensures that fault information is captured synchronously and remains stable for observation and debugging. All access decision logic operates without reliance on the clock, allowing immediate response to memory access requests.

9.5 Synthesizable Design Style

The design follows a fully synthesizable coding style, using only constructs supported by standard synthesis tools. The implementation avoids non-synthesizable features such as delays, file I/O, or simulation-only system tasks within the design modules. This ensures that the SMAC can be mapped to FPGA or ASIC hardware if required.

9.6 Latch-Free Design

All combinational always blocks explicitly assign default values to outputs to prevent unintended latch inference. This ensures predictable behavior and improves reliability. The absence of latches is critical for maintaining timing correctness and simplifying verification.

9.7 Blocking vs Non-Blocking Assignments

Blocking assignments (=) are used in combinational logic to model immediate signal evaluation within always-@(*) blocks. Non-blocking assignments (<=) are used in sequential logic within clocked always blocks, particularly in the fault logger module. This distinction ensures correct simulation semantics and matches best practices for hardware design.

10. Simulation & Verification

This section validates the functional correctness of the Secure Memory Access Controller (SMAC) using ModelSim. Verification is performed through directed test cases, waveform analysis, and console output inspection to ensure that privilege-based and region-based execution permissions are enforced correctly.

10.1 Simulation Environment

The SMAC design was simulated using ModelSim with the following setup:

HDL: Verilog

Design Style: Modular and synthesizable

Verification Method: Testbench-driven simulation

Clock Frequency: 100 MHz (10 ns period)

Inputs Applied: Address, privilege mode, access type (Execute)

Outputs Observed: mem_enable, access_fault, fault_address

All test cases were executed using a dedicated testbench without modifying the design modules.

10.2 Test Case Description

The objective of the simulation is to verify that:

- Execution is allowed only in regions where execute permission is defined.
- Privilege violations correctly raise access faults.
- Kernel mode has full execution access.
- Fault address logging operates correctly on illegal access.

10.3 Test Case Table

Test Case	Privilege Mode	Address	Access Type	Expected Result
TC1	USER (00)	0x1000	Execute	Allow

TC2	USER (00)	0x3000	Execute	Fault
TC3	SUPERVISOR (01)	0x5000	Execute	Fault
TC4	KERNEL (10)	0x7000	Execute	Allow

10.4 Simulation Results

The simulation results confirm that the SMAC correctly enforces execution permissions based on both privilege level and memory region.

TC1: User mode execution in Code region was allowed as expected.

TC2: User mode execution attempt in Data region resulted in an access fault.

```

initial begin
    clk = 0;

    // -----
    // USER execute CODE (ALLOW)
    // -----
    address = 16'h1000;
    current_mode = 2'b00;    // USER
    mem_read = 0;
    mem_write = 0;
    mem_execute = 1;
    #10;

    // -----
    // USER execute DATA (FAULT)
    // -----
    address = 16'h3000;
    current_mode = 2'b00;
    mem_execute = 1;
    #10;

```

TC3: Supervisor mode execution attempt in OS region was blocked due to lack of execute permission.

TC4: Kernel mode execution in Kernel region was allowed, demonstrating full privilege access.

```

// -----
// SUPERVISOR execute OS (FAULT)
// -----
address = 16'h5000;
current_mode = 2'b01;    // SUPERVISOR
mem_execute = 1;
#10;

// -----
// KERNEL execute KERNEL (ALLOW)
// -----
address = 16'h7000;
current_mode = 2'b10;    // KERNEL
mem_execute = 1;
#10;

```

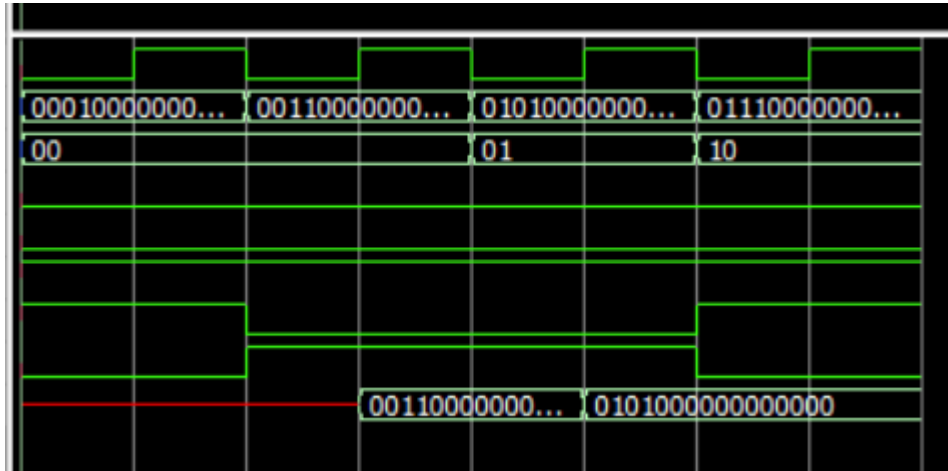
All observed outputs matched the expected behavior defined by the protection policy.

10.5 Waveform Analysis

The waveform analysis clearly demonstrates the internal behavior of the SMAC during each memory request:

- mem_execute asserts during execution requests.
- current_mode reflects the active privilege level.
- mem_enable goes high only for authorized execution.
- access_fault asserts for illegal execution attempts.
- fault_address captures the offending memory address during fault conditions.

This confirms that access control decisions are made combinatorially, while fault logging is synchronized with the system clock.



10.6 Console Output Verification

In addition to waveform analysis, textual verification was performed using \$display and \$monitor statements in the testbench. This provided a tabular, time-stamped view of access decisions, improving traceability and easing result validation.

```
VSIM 3> run -all
#
# -----
# Time | Mode | Address | R W X | Enable | Fault | Fault Address
# -----
# 0 | 00 | 0x1000 | 0 0 1 | 1 | 0 | 0xxxxx
# 10 | 00 | 0x3000 | 0 0 1 | 0 | 1 | 0xxxxx
# 15 | 00 | 0x3000 | 0 0 1 | 0 | 1 | 0x3000
# 20 | 01 | 0x5000 | 0 0 1 | 0 | 1 | 0x3000
# 25 | 01 | 0x5000 | 0 0 1 | 0 | 1 | 0x5000
# 30 | 10 | 0x7000 | 0 0 1 | 1 | 0 | 0x5000
# -----
# Simulation Completed
# ** Note: $stop : C:/Users/User/Desktop/Uni/Computer Architecture/verilogLab/SMAC_TB.v(82)
# Time: 40 ps Iteration: 0 Instance: /SMAC_TB
# Break in Module SMAC_TB at C:/Users/User/Desktop/Uni/Computer Architecture/verilogLab/SMAC_TB.v line 82
```

11. Results & Discussion

The simulation and verification results demonstrate that the Secure Memory Access Controller (SMAC) functions correctly according to its design objectives.

The system successfully blocked all illegal memory access attempts based on privilege level and memory region. User-mode and Supervisor-mode programs were prevented from executing code in protected regions, while Kernel-mode operations were granted full access as intended.

Execution protection was verified by enforcing execute permissions only in designated code regions. Attempts to execute instructions from non-executable regions resulted in immediate access faults, confirming that execute permission enforcement is working correctly.

Fault handling behavior was also validated. On each illegal access attempt, the SMAC correctly asserted the `access_fault` signal and logged the offending memory address. The captured fault address matched the attempted access address in all test cases, confirming the correctness of the fault logging mechanism.

Overall, the observed results align with the defined protection policy and demonstrate reliable, deterministic behavior suitable for hardware-level security enforcement.

12. Limitations & Future Enhancements

While the current SMAC implementation fulfills its core security objectives, several enhancements can be incorporated to improve scalability and real-world applicability.

Currently, memory regions are statically defined. A future enhancement would include configurable region registers, allowing the operating system to dynamically program memory protection policies at runtime.

To strengthen protection against peripheral-initiated attacks, DMA device identification can be added. This would enable the SMAC to apply different access policies based on device IDs, similar to IOMMU behavior.

The current design operates at region granularity. Page-level protection would provide finer control, improving compatibility with modern virtual memory systems.

Integration with an IOMMU (Input-Output Memory Management Unit) would further enhance protection against DMA attacks by isolating peripheral memory accesses from system RAM.

Finally, formal verification techniques such as property checking and model checking can be applied to mathematically prove the correctness of access control and fault handling logic.

13. Conclusion

This project successfully designed and implemented a Secure Memory Access Controller (SMAC) that enforces privilege-based and region-based memory protection at the hardware level.

The system ensures that only authorized execution requests are permitted, while illegal accesses are blocked and reported through fault signaling. By performing access checks in hardware, the design eliminates reliance on software-only enforcement mechanisms and significantly improves system security.

The SMAC architecture reflects concepts used in real-world processors and security subsystems, demonstrating practical relevance to modern computer architecture. The project meets all stated objectives and provides a strong foundation for further enhancements in secure system design.

14. References

[ARM Ltd., ARM® Architecture Reference Manual, ARM Documentation.](#)

[RISC-V Foundation, The RISC-V Privileged Architectures Specification.](#)

[William Stallings, Computer Organization and Architecture, Pearson Education.](#)

[Intel Corporation, DMA Attacks and Mitigation Techniques, Security White Papers.](#)

[Mark Seaborn et al., Thunderclap: Exploring Vulnerabilities in DMA Protection, Google Research.](#)

[Microsoft, Kernel DMA Protection Overview, Microsoft Documentation.](#)